

databricks_de_practice.py — Line-by-Line Explained

Every cell and line of the medallion/Delta/windows notebook, explained

medallion · clean/dedup · broadcast join · star schema · MERGE · windows · time travel/OPTIMIZE

A detailed walkthrough of the main practice notebook (the medallion / Delta / window-functions one). Each cell's code is explained line by line, with the concepts introduced where they appear.

Step 0 — Setup (imports & DBFS paths)

```
from pyspark.sql import functions as F, Window
```

- **functions as F** — Spark's built-in column functions (`F.col`, `F.sum`, `F.to_date`, ...).
- **Window** — used to define **window functions** (row_number, running totals) later.

```
BASE = "/tmp/de_practice"
RAW = f"{BASE}/raw" # bronze (raw Parquet)
SILVER = f"{BASE}/silver" # cleaned
GOLD = f"{BASE}/gold" # star schema (Delta)
```

- Path variables for the **medallion zones**: raw (as-ingested), silver (cleaned), gold (modeled). Keeping them as variables means you change the location in one place.

```
dbutils.fs.rm(BASE, recurse=True)
```

- `dbutils.fs.rm(path, recurse=True)` deletes the whole working folder so each run starts clean (makes the notebook **idempotent** — safe to re-run top to bottom).
-

Step 1 — Generate sample data & write Parquet (Bronze)

```
customers = [(i, f"Customer {i}", random.choice(REGIONS) if random.random()>0.05 else None) for i in range(1,201)]
```

- A **list comprehension** building 200 customer tuples. `random.choice(REGIONS)` picks a region; `if random.random()>0.05 else None` leaves ~5% region **null** on purpose (so you clean it later).

```
orders=[]; oid=100000
for _ in range(10000):
    oid+=1; ...
    amount=round(qty*price,2) if random.random()>0.03 else None
    orders.append((oid, random.randint(1,210), ...))
orders += random.sample(orders, 150) # inject duplicates
```

- Builds 10,000 orders; ~3% **have null amounts**; `customer_id` can be up to 210 (some point to **non-existent** customers — bad refs). `orders += random.sample(orders,150)` appends 150

duplicate rows. This messiness is intentional — it's what ETL cleans.

```
cust_df = spark.createDataFrame(customers, ["customer_id","name","region"])
...
ord_df = spark.createDataFrame(orders, ["order_id",...,"order_ts"])
```

- `spark.createDataFrame(data, columns)` turns each Python list into a distributed DataFrame.

```
ord_df.write.mode("overwrite").parquet(f"{RAW}/orders")
```

- Writes the raw data as **Parquet** to the bronze zone. `.mode("overwrite")` = replace on re-run.

```
display(dbutils.fs.ls(f"{RAW}/orders"))
```

- Lists the written `part-*.parquet` files — your sample Parquet.

Step 2 — Read & explore

```
orders = spark.read.parquet(f"{RAW}/orders")
```

- Reads the Parquet back (schema inferred from the file).

```
orders.printSchema()
print("row count:", orders.count())
display(orders.limit(10))
```

- `printSchema()` — columns + types. `count()` — total rows (~10,150 incl. dupes). `limit(10)` — a 10-row preview; `display` renders it.

```
display(orders.groupBy("status").count())
print("null amounts:", orders.filter(F.col("amount").isNull()).count())
```

- `groupBy("status").count()` — how many completed/cancelled/pending (see the mess).
- `filter(F.col("amount").isNull())` — rows with a null amount; `.count()` how many.

Step 3 — Bronze → Silver (clean the data)

```
w = Window.partitionBy("order_id").orderBy(F.col("order_ts").desc())
```

- Defines a **window**: group rows by `order_id`, ordered by timestamp **descending**. Used to pick the latest row per order (deduplication).

```
silver = (orders
    .withColumn("amount", F.col("amount").cast("double"))
    .filter((F.col("status")== "completed") & (F.col("amount") > 0))
    .withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
    .join(F.broadcast(customers.select("customer_id")), "customer_id")
    .join(F.broadcast(products.select("product_id")), "product_id")
    .withColumn("order_date", F.to_date("order_ts")))
```

- `.withColumn("amount", ...cast("double"))` — ensure amount is numeric.
- `.filter((status=="completed") & (amount>0))` — keep only valid completed orders. `&` is Spark's AND; each condition needs parentheses.
- `F.row_number().over(w)` — assign 1,2,3... within each `order_id` (latest first). `filter("rn=1")` keeps just the latest → **dedup**. `.drop("rn")` removes the helper column.
- `.join(F.broadcast(customers...), "customer_id")` — inner join to valid customers only (drops **orphan** references). `F.broadcast(...)` ships the small table to every executor to avoid a shuffle (a **broadcast join** — fast).
- `.withColumn("order_date", F.to_date("order_ts"))` — derive a DATE for partitioning.

```
print("raw:", orders.count(), " -> clean:", silver.count())
silver.write.format("delta").mode("overwrite").partitionBy("order_date").save(f"
{SILVER}/orders")
```

- Prints the before/after counts (you'll see the drop from cleaning).
- Writes **Delta**, **partitioned by** `order_date` so later date-filtered queries prune folders.

Step 4 — Gold: dimensional model (star schema)

```
silver = spark.read.parquet(f"{SILVER}/orders")
```

- (If Delta) read the cleaned data back to build gold tables from it.

```
dim_customer = customers.withColumn("region", F.coalesce("region", F.lit("Unknown")))
```

- `F.coalesce("region", F.lit("Unknown"))` — replace null regions with the string "Unknown" (returns the first non-null). A clean dimension has no nulls in display fields.

```
dim_date = (silver.select("order_date").distinct()
              .withColumn("year", F.year("order_date"))
              .withColumn("month", F.month("order_date"))
              .withColumn("day", F.dayofmonth("order_date")))
```

- Builds a **date dimension**: distinct dates plus derived `year / month / day` for easy grouping.

```
fact_orders =
silver.select("order_id","customer_id","product_id","order_date","quantity","amount")
```

- The **fact table**: the measurable events (amount, quantity) + foreign keys. Grain = one order.

```
for name, df in [("dim_customer",dim_customer), ...]:
    df.write.format("delta").mode("overwrite").save(f"{GOLD}/{name}")
```

- Loops over the 4 tables and writes each as **Delta** into the gold zone.

Step 5 — Register Delta tables & query with SQL

```
spark.sql("CREATE DATABASE IF NOT EXISTS de_practice")
for name in [...]:
    spark.sql(f"CREATE TABLE IF NOT EXISTS de_practice.{name} USING DELTA LOCATION
'{GOLD}/{name}'")
```

- Creates a database and registers each gold folder as a **Delta table** (queryable by name). Using `LOCATION` makes them **external** tables pointing at your files.

```
display(spark.sql("""
SELECT c.region, ROUND(SUM(f.amount),2) revenue, COUNT(*) orders
FROM de_practice.fact_orders f JOIN de_practice.dim_customer c ON c.customer_id=f.customer_id
GROUP BY c.region ORDER BY revenue DESC"""))
```

- A classic **star-join**: sum the fact (`amount`) grouped by a dimension attribute (`region`). This is the shape BI tools use.

Step 6 — MERGE (upsert) — incremental/CDC

```
from delta.tables import DeltaTable

updates = spark.createDataFrame([...existing id..., ...new id...], [...])
```

- A small batch with one **existing** order (to update) and one **new** order (to insert).

```
fact = DeltaTable.forPath(spark, f"{GOLD}/fact_orders")
(fact.alias("t").merge(updates.alias("s"), "t.order_id = s.order_id")
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .execute())
```

- `DeltaTable.forPath` wraps the Delta table for MERGE.
- `.merge(source, "t.order_id=s.order_id")` — upsert on the key.
- `whenMatchedUpdateAll` updates the existing order; `whenNotMatchedInsertAll` inserts the new one — atomically. This is the everyday **incremental load** pattern.

Step 7 — Window functions

```
w1 = Window.partitionBy("customer_id").orderBy(F.col("amount").desc())
display(fact.withColumn("rn", F.row_number().over(w1)).filter("rn <= 3") ...)
```

- **Top-N per group:** `row_number()` numbers each customer's orders by amount (desc); `rn <= 3` keeps each customer's **top 3**.

```
w2 = Window.partitionBy("customer_id").orderBy("order_date") \
  .rowsBetween(Window.unboundedPreceding, Window.currentRow)
display(fact.withColumn("running_total", F.sum("amount").over(w2)) ...)
```

- **Running total:** `sum(amount)` over a window ordered by date, framed from the start (`unboundedPreceding`) to the current row → cumulative spend per customer over time.

Step 8 — Analytics

```
top_products = (fact.join(F.broadcast(dim_product), "product_id")
  .groupBy("product_name", "category")
  .agg(F.round(F.sum("amount"), 2).alias("revenue"), F.sum("quantity").alias("units"))
  .orderBy(F.col("revenue").desc()))
```

- Join fact to the (small, **broadcast**) product dimension, then `.agg(...)` computes revenue and units per product. `.alias(...)` names the output columns.

```
daily =
fact.groupBy("order_date").agg(F.round(F.sum("amount"),2).alias("revenue")).orderBy("order_date"
)
```

- Daily revenue trend — group by date, sum amount.

Step 9 — Delta features (time travel, OPTIMIZE, Z-ORDER)

```
display(spark.sql("DESCRIBE HISTORY
de_practice.fact_orders").select("version","timestamp","operation"))
```

- **DESCRIBE HISTORY** shows every version of the Delta table (the transaction log) — writes, merges, optimizes — with version numbers and operations. The basis for **time travel** and audit.

```
v0 = spark.read.format("delta").option("versionAsOf", 0).load(f"{GOLD}/fact_orders")
print("rows at version 0:", v0.count())
```

- **.option("versionAsOf", 0)** reads the table **as it was at version 0** (before the MERGE) — Delta **time travel**. Great for recovering from bad writes or auditing.

```
spark.sql(f"OPTIMIZE delta.`{GOLD}/fact_orders` ZORDER BY (customer_id)")
```

- **OPTIMIZE** compacts many small files into fewer big ones (fixes the small-files problem). **ZORDER BY (customer_id)** co-locates rows with similar **customer_id** so queries filtering on it read less data (**data skipping**).

```
# spark.sql(f"VACUUM delta.`{GOLD}/fact_orders` RETAIN 168 HOURS")
```

- **VACUUM** (commented out) removes old, unreferenced files. Default retention is **7 days** (protects time travel) — don't lower it in production without care.

Step 10 — Recap (markdown)

Summarizes the whole flow: wrote/read **Parquet**, cleaned Bronze→Silver, built a **Gold star schema**, used **Delta** (MERGE, time travel, OPTIMIZE/Z-ORDER), and **window functions** — the core skills a Databricks data-engineering role tests.

The concepts you practiced (interview-ready)

- **Medallion architecture**: raw → clean → curated zones.

- **Cleaning:** cast types, filter, **dedup with** `row_number()`, referential integrity via joins.
- **Broadcast join:** ship a small table to executors to avoid a shuffle.
- **Partitioning + Parquet/Delta:** faster, cheaper reads via pruning.
- **Star schema:** fact + dimensions; grain = one order.
- **Delta MERGE:** upsert / incremental / CDC.
- **Window functions:** top-N per group, running totals.
- **Delta features:** time travel, OPTIMIZE, Z-ORDER, VACUUM.